

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA43

Name: T. Nikhil Kumar Reddy

Reg-No: 192372024

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS:

Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly.

Questions:

1. Identify appropriate **HTML5 semantic elements** missing in the structure.
2. Modify the structure to make it **standards-compliant**.
3. Interpret the **HTTP request generated** during form submission.
4. Analyze the **HTTP response cycle** from server to browser.
5. Suggest improvements for **usability and accessibility**.

Question 1: Online Symposium Portal

Given:

html

```
<form action="/register" method="POST">
```

```
  <input type="text" name="name">
```

```
  <input type="email" name="email">
```

```
  <button>Submit</button>
```

```
</form>
```

1. Identify appropriate HTML5 semantic elements missing in the structure

- No `<!DOCTYPE html>`, `<html>`, `<head>`, or `<body>` wrapper — the snippet is a fragment, not a full document.
- No `<label>` elements associated with the inputs — screen readers cannot identify what each field represents.
- No `<fieldset>/<legend>` to group related form fields logically.
- No `<main>`, `<section>`, or `<header>` to semantically structure the page (e.g., a heading for "Symposium Registration").
- Missing required `id` attributes on inputs, which HTML5 supports for validation and label association.
- No `<h1>/<h2>` heading identifying the purpose of the form.

2. Modify the structure to make it standards-compliant

html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>Symposium Registration</title>
```

```
</head>
```

```
<body>
```

```
  <header>
```

```
<h1>Online Symposium Registration</h1>

</header>

<main>

  <section>

    <form action="/register" method="POST">

      <fieldset>

        <legend>Registration Details</legend>

        <label for="name">Full Name:</label>

        <input type="text" id="name" name="name" required>

        <label for="email">Email Address:</label>

        <input type="email" id="email" name="email" required>

        <button type="submit">Submit</button>

      </fieldset>

    </form>

  </section>

</main>

</body>

</html>
```

3. Interpret the HTTP request generated during form submission

- Since method="POST" is used, submitting the form generates an HTTP request like:

POST /register HTTP/1.1

Host: college-symposium.edu

Content-Type: application/x-www-form-urlencoded

Content-Length: 34

name=John+Doe&email=john%40mail.com

- Key points:
 - The **request line** specifies method (POST), path (/register), and HTTP version.
 - **Content-Type** defaults to application/x-www-form-urlencoded (form data is URL-encoded and placed in the request body, not the URL).
 - **Content-Length** indicates the byte size of the encoded body.
 - Unlike GET, the submitted data (name, email) does not appear in the URL — it is sent in the request body.

4. Analyze the HTTP response cycle from server to browser

1. The server (e.g., a servlet, PHP, or Node.js backend) receives the POST request and parses the body to extract name and email.
2. The server validates/stores the data (e.g., inserts into a database).
3. The server sends back an HTTP response, for example:

HTTP/1.1 200 OK

Content-Type: text/html

Content-Length: 128

<html><body><h2>Registration Successful</h2></body></html>

or, following the **Post/Redirect/Get (PRG)** pattern:

HTTP/1.1 303 See Other

Location: /registration-success

4. The browser receives the response:
 - If status is 200 OK with an HTML body, the browser renders it directly.
 - If status is a redirect (303/302), the browser automatically issues a new **GET** request to the Location URL, preventing duplicate form resubmission if the user refreshes the page.
5. Finally, the browser parses and renders the confirmation page to the user.

5. Suggest improvements for usability and accessibility

- Add `<label for="">` for every input so screen readers announce field purposes correctly.
- Use placeholder attributes as supplementary hints (not a replacement for labels).
- Add **client-side validation** using HTML5 attributes (`required`, `pattern`, `type="email"`) to give instant feedback.
- Provide **visible error messages** near invalid fields rather than relying only on browser default tooltips.
- Ensure sufficient **color contrast** and **keyboard navigability** (tab order) for accessibility (WCAG compliance).
- Add **ARIA attributes** (`aria-required="true"`, `aria-describedby`) for enhanced screen-reader support.
- Show a **loading indicator** or disable the submit button after click to prevent duplicate submissions.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
<head>
  <title>Company</title>
</head>
<body>
  <div>
    <h1>Welcome
    <p>About us
  </div>
</body>
</html>
```

Questions:

1. Identify **improper nesting and missing closing tags**.
2. Analyze how different browsers may interpret this structure.
3. Identify **missing semantic elements** (e.g., header, section).
4. Provide a **corrected W3C-compliant HTML structure**.
5. Suggest techniques to ensure **cross-browser compatibility**.

1. Identify improper nesting and missing closing tags

- Missing `<!DOCTYPE html>` declaration — without it, browsers may render in **quirks mode** instead of standards mode.
- `<h1>Welcome` — missing closing `</h1>` tag.
- `<p>About us` — missing closing `</p>` tag.

- `<div>` is closed, but since `<h1>` and `<p>` inside it were never explicitly closed, their closure is left to browser auto-correction, causing improper/ambiguous nesting.

2. Analyze how different browsers may interpret this structure

- Since there's no DOCTYPE, some older or non-standard browsers may render the page in **quirks mode**, applying non-standard CSS box models and layout rules — this alone can cause visible inconsistency across browsers.
- Modern browsers (Chrome, Firefox, Edge) generally auto-close `<h1>` when `<p>` starts, and auto-close `<p>` when `</div>` is encountered — but the exact error-recovery algorithm can subtly differ between rendering engines (Blink, Gecko, WebKit), especially with more deeply nested or malformed markup.
- This inconsistent auto-correction is exactly why the "inconsistent layout across browsers" symptom occurs — each browser's HTML parser silently "fixes" the broken markup differently.

3. Identify missing semantic elements (e.g., header, section)

- No `<meta charset="UTF-8">` for character encoding declaration.
- No `<header>` element to wrap the site branding/navigation.
- No `<main>` element for the primary content.
- No `<section>` or `<article>` to group "About Us" content meaningfully.
- No `<footer>` for closing page information.
- No `<nav>` if navigation links are expected on a company website.

4. Provide a corrected W3C-compliant HTML structure

html

`<!DOCTYPE html>`

`<html lang="en">`

`<head>`

`<meta charset="UTF-8">`

`<meta name="viewport" content="width=device-width, initial-scale=1.0">`

`<title>Company</title>`

`</head>`

`<body>`

```
<header>

  <h1>Welcome</h1>

</header>

<main>

  <section>

    <p>About us</p>

  </section>

</main>

<footer>

  <p>&copy; 2026 Company Name</p>

</footer>

</body>

</html>
```

5. Suggest techniques to ensure cross-browser compatibility

- Always include `<!DOCTYPE html>` to force **standards mode** rendering consistently.
- Validate HTML using the **W3C Markup Validator** to catch unclosed/misnested tags before deployment.
- Use a **CSS reset or normalize.css** to eliminate default styling differences between browsers.
- Use **vendor prefixes** or tools like Autoprefixer for CSS properties that need them.
- Test consistently using **cross-browser testing tools** (e.g., BrowserStack, LambdaTest).
- Use **feature detection** (via Modernizr or `@supports` in CSS) instead of browser-sniffing.
- Avoid deprecated/non-standard tags; use semantic HTML5 elements uniformly.
- Keep CSS layout techniques modern and well-supported (Flexbox/Grid) with fallback considerations for older browsers if required.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify issues in the **form configuration**.
2. Analyze why the **HTTP request is not triggered**.
3. Interpret the role of **GET method in this scenario**.
4. Propose a **corrected implementation**.
5. Suggest improvements for **secure data transmission**.

1. Identify issues in the form configuration

- **method="GET"** is used for a login form — this is a major issue because credentials (username/password) will be exposed in the URL query string, not appropriate for sensitive data.
- **<button type="button">** — this is the critical functional bug. A type="button" button does **not** submit the form by default; it only triggers submission if a JavaScript onclick handler explicitly calls form.submit(). Since no such script is shown, clicking "Login" does **nothing**.
- No required attributes on the input fields to prevent empty submissions.
- No CSRF protection or HTTPS consideration mentioned for secure transmission.

2. Analyze why the HTTP request is not triggered

- In HTML forms, only a button with **type="submit"** (or a plain <button> with no type, which defaults to submit) triggers the form's default submission behavior.
- Here, type="button" explicitly overrides the default and tells the browser: *"this button is a generic button, not a submit control."*
- As a result, clicking it produces **no HTTP request at all** unless custom JavaScript intervenes — this is exactly why "the web application fails to send form data to the server."

3. Interpret the role of the GET method in this scenario

- Even if the button type were corrected, using **GET** for login would append data to the URL, e.g.:

GET /submit?username=admin&password=1234 HTTP/1.1

- This is **functionally working** but **insecure**:
 - Credentials appear in the browser address bar, history, and server access logs.
 - Data may be cached by browsers/proxies.
 - Sensitive information can leak via the Referer header.

- So GET is technically capable of submitting the data as a query string, but it's the **wrong method choice** for any form handling confidential information like passwords.

4. Propose a corrected implementation

html

```
<form action="/submit" method="POST">

<label for="username">Username:</label>

<input type="text" id="username" name="username" required>


<label for="password">Password:</label>

<input type="password" id="password" name="password" required>


<button type="submit">Login</button>

</form>
```

- Changed method="GET" → method="POST" to hide credentials from the URL.
- Changed type="button" → type="submit" so clicking the button actually triggers form submission.
- Added <label> and required attributes for accessibility and basic validation.

5. Suggest improvements for secure data transmission

- Serve the form and process it only over **HTTPS (TLS)** so data is encrypted in transit — POST alone hides data from the URL but does not encrypt it.
- Hash and salt passwords on the **server side** before storing them (e.g., bcrypt/Argon2); never store or log plaintext passwords.
- Implement **CSRF tokens** in the form to prevent cross-site request forgery attacks.
- Set **autocomplete="off"** or appropriate autocomplete hints on the password field where relevant, based on security policy.
- Use **rate limiting / account lockout / CAPTCHA** after repeated failed login attempts.
- Enforce **secure, HttpOnly, SameSite** cookies for session tokens post-login.
- Consider **multi-factor authentication (MFA)** for an added layer of protection.

Code of Conduct:

I **T. Nikhil Kumar Reddy / 192372024** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: T. Nikhil Kumar Reddy

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand